📘 Introduction to Data Science — Complete Study Material

> ***Course Code:*** *BCSE2C05 / BCCS2C01 / BMNC2C01* | ***Semester:*** *II This material covers every concept from the syllabus in simple language so you can solve all question papers confidently.*

🔷 UNIT 1: INTRODUCTION TO DATA SCIENCE

1.1 What is Data Science?

Data Science is a field that combines statistics, mathematics, programming, and domain knowledge to extract meaningful insights from data.

In simple words: You have raw data → you clean it → analyse it → build models → make predictions or decisions.

Key Components:

- **Mathematics & Statistics** — Probability, distributions, hypothesis testing.
- **Programming** — Python, R, SQL for data manipulation.
- **Domain Knowledge** — Understanding the business problem (healthcare, finance, etc).
- **Machine Learning** — Algorithms that learn from data.
- **Visualization** — Communicating findings through charts and graphs.

1.2 What is Big Data?

Big Data = Datasets that are so large, fast, and complex that traditional tools (like Excel or MySQL) can't handle them.

Think of it this way: Excel can handle thousands of rows. But what if you have 10 billion rows of GPS data from Uber drivers streaming in every second? That's Big Data.

The 5 V's of Big Data

V	What it means	Real Example	Why it matters
Volume	Massive amount of data	Facebook: 4+ petabytes/day (1 PB = 10 lakh GB)	Need distributed storage (Hadoop, cloud)
Velocity	Speed of data generation	Stock market: millisecond updates	Need real-time processing (Kafka, Spark)
Variety	Different types of data	Text + images + videos + GPS + audio	Need tools that handle all formats
Veracity	Quality and trustworthiness	Twitter bots posting fake news	Need data validation and cleaning
Value	Usefulness after processing	Netflix recommendations → saves $1B/year	Raw data is useless; insights are valuable

Tools for Big Data:

- **Storage:** Hadoop HDFS, Amazon S3, Google Cloud Storage
- **Processing:** Apache Spark, MapReduce
- **Streaming:** Apache Kafka, Apache Flink

1.3 Evolution of Data Science

Era	What Happened
1960s–70s	Statistics and early computing. Data stored in flat files.

1980s	Relational databases invented (SQL). Data became queryable.
1990s	Data Mining emerged — finding patterns in databases. Data warehousing.
2000s	Big Data era — Hadoop, distributed computing. Google's MapReduce paper (2004).
2010s	Machine Learning & Deep Learning boom. Scikit-learn, TensorFlow.
2020s	AI, Large Language Models (GPT), AutoML, real-time analytics. Democratization of data science.

1.4 Data Science Lifecycle

This is the step-by-step process every data science project follows:

```
1. Problem Definition
       ↓
2. Data Collection
       ↓
3. Data Cleaning
       ↓
4. Exploratory Data Analysis (EDA)
       ↓
5. Feature Engineering
       ↓
6. Model Building
       ↓
7. Model Evaluation
       ↓
8. Deployment & Monitoring
```

Each Phase Explained:

1. Problem Definition

- Define WHAT you want to solve. Be specific.
- × Bad: "Improve sales." ✅ Good: "Predict which customers will churn in the next 30 days."

2. Data Collection

- Gather data from databases, APIs, surveys, web scraping, sensors.
- Sources: CSV files, SQL databases, REST APIs, IoT devices.

3. Data Cleaning

- Real data is MESSY. You'll spend 60-80% of time here.
- Tasks: Handle missing values, remove duplicates, fix typos, standardize formats.

4. Exploratory Data Analysis (EDA)

- Understand relationships and patterns using statistics and plots.
- Compute mean, median, mode. Draw histograms, boxplots, scatter plots.

5. Feature Engineering

- Create new useful columns from existing data.
- Example: From `Joining_Date`, create `Tenure_Years = Today − Joining_Date`.

6. Model Building

- Choose and train ML algorithms: Linear Regression, Decision Tree, k-NN, K-Means, etc.
- Split data: 80% training, 20% testing.

7. Model Evaluation

- Check how good the model is using metrics: Accuracy, Precision, Recall, F1-Score, RMSE.

8. Deployment & Monitoring

- Put the model into production (as an API, dashboard, or app).
- Monitor for data drift — is the model still accurate over time?

1.5 Data Mining

Data Mining = The process of discovering hidden patterns and useful knowledge from large datasets.

Two Types:

Type	What it does	Question it answers	Techniques	Example
Descriptive	Summarizes past data	"What happened?"	Clustering, Association Rules, Summarization	"60% of laptop buyers also buy laptop bags"
Predictive	Predicts future outcomes	"What will happen?"	Classification, Regression, Time-series	"Customer X has 78% chance of leaving"

How they work together:

Descriptive mining finds patterns → Predictive mining uses those patterns to forecast.

1.6 Data Science Roles

Role	What they do	Tools they use
Data Scientist	Analyses data, builds ML models, extracts insights	Python, R, SQL, Scikit-learn
Data Engineer	Builds & maintains data pipelines and infrastructure	Spark, Hadoop, Airflow, SQL
Data Analyst	Explores data, creates reports & dashboards	Excel, Power BI, Tableau, SQL
ML Engineer	Deploys & scales ML models to production	Docker, Kubernetes, FastAPI
Business Analyst	Translates business needs into data requirements	Excel, communication skills

---

1.7 Applications of Data Science

Industry	Application	How Data Science Helps
Healthcare	Disease prediction, drug discovery	ML models diagnose diseases from scans
Finance	Fraud detection, credit scoring	Algorithms flag suspicious transactions in real-time
E-Commerce	Product recommendations	"Customers who bought X also bought Y"
Transportation	Route optimization, demand prediction	Uber's surge pricing, Google Maps ETA
Agriculture	Crop yield prediction	Satellite + weather data for precision farming
Education	Student performance prediction	Identify at-risk students early
Social Media	Sentiment analysis, ad targeting	Analyse millions of posts for trends
Sports	Player performance analytics	Moneyball approach — data-driven team building

---

1.8 Data Collection Strategies

How do we actually GET data?

Strategy	What it is	Example	Pros	Cons
Surveys	Questionnaires to people	Google Forms feedback survey	Direct, specific	Small sample, response bias
Interviews	One-on-one conversations	Interviewing patients for medical study	Deep insights	Time-consuming, not scalable
Observation	Watching & recording	Counting footfall in a mall	Authentic behavior	Observer bias
Experiments	Controlled tests	A/B testing a website button	Causal conclusions	Expensive, ethical concerns
Web Scraping	Extracting data from websites	Scraping prices from Flipkart	Large-scale, automated	Legal issues, site structure changes
APIs	Programmatic data access	Twitter API for tweets	Structured, reliable	Rate limits, authentication
Existing Data	Pre-collected datasets	Census data, Kaggle datasets	Free, readily available	May not fit your exact need
Sensors/IoT	Automated from devices	Weather stations, fitness bands	Continuous, real-time	Hardware costs, noise

Primary vs Secondary Data:

	Primary Data	Secondary Data
Collected by	You (first-hand)	Someone else (pre-existing)
Source	Surveys, experiments, interviews	Government reports, journals, Kaggle
Cost	Expensive	Cheap or free
Fit	Perfectly fits your need	May not fit exactly
Example	Conducting a customer survey	Using Census data for analysis

---

1.9 Sources of Data

1. Time Series Data

- **What:** Data collected at regular time intervals.
- **Example:** Daily stock prices, hourly temperature readings, monthly sales figures.
- **Challenges:** Handling trends, seasonality, missing timestamps.
- **Tools:** Pandas `DatetimeIndex`, ARIMA models.

2. Transactional Data

- **What:** Records of business events/transactions.
- **Example:** E-commerce orders, bank transfers, POS receipts.
- **Challenges:** High volume, real-time processing, data integrity.

3. Biological Data

- **What:** Data from living organisms.
- **Example:** DNA sequences, protein structures, patient health records.
- **Challenges:** Extremely large (human genome = 3 billion base pairs), privacy concerns (HIPAA).

4. Spatial Data

- **What:** Data tied to geographic locations.
- **Example:** GPS coordinates, satellite images, GIS maps.
- **Challenges:** Different coordinate systems, large file sizes, real-time processing.
- **Tools:** Folium, GeoPandas, Google Maps API.

5. Social Network Data

- **What:** Data from social media interactions.
- **Example:** Tweets, Facebook posts, LinkedIn connections, Instagram likes.
- **Challenges:** Unstructured text, fake accounts/bots, API rate limits, privacy.

Data Evolution

Data is constantly evolving — new sources emerge (IoT sensors, wearables), formats change, volumes grow exponentially. Modern data systems must be flexible enough to handle this.

🔷 UNIT 2: EXPLORATORY DATA ANALYTICS

2.1 Data Objects and Attribute Types

What is a Data Object?

A **data object** represents a single entity in your dataset — one row in a table.

- In a student database → each student is a data object.
- In a hospital → each patient is a data object.

What is an Attribute?

An **attribute** (also called a feature or column) describes a property of the data object.

- Student's `Name`, `Age`, `CGPA` — each is an attribute.

Types of Attributes:

```
Attributes
├── Qualitative (Categorical)
│   ├── Nominal — No order (Color, Gender)
│   ├── Ordinal — Has order (Rating: Poor < Good < Excellent)
│   └── Binary — Only 2 values (Yes/No, True/False)
│       ├── Symmetric (both values equally important)
│       └── Asymmetric (one value is more significant)
└── Quantitative (Numeric)
    ├── Discrete — Countable (Number of students: 30, 31, 32)
    └── Continuous — Measurable (Height: 5.5 ft, Temperature: 36.7°C)
```

Detailed Breakdown:

Nominal — Just labels, no ordering.

- Examples: City (Delhi, Mumbai, Pune), Blood Type (A, B, AB, O), Color (Red, Blue).
- You CAN'T say: Delhi > Mumbai. They're just different categories.
- Operations: Only = or ≠(equality check).

Ordinal — Categories WITH a meaningful order, but differences aren't measurable.

- Examples: Education (High School < Bachelor's < Master's < PhD), Star Rating (★ < ★★ < ★★★).
- You CAN say: PhD > Bachelor's. But "PhD − Bachelor's = 2 units" doesn't make sense.
- Operations: =, ≠, <, > (can compare).

Binary — Special case of nominal with exactly 2 values.

- **Symmetric Binary:** Both values are equally important.
  - Example: Gender (Male/Female) — both are equally significant.
  - When measuring similarity, both 0-0 and 1-1 matches count.
  - **Formula:** Simple Matching Coefficient = $(f_{11} + f_{00}) / (f_{11} + f_{10} + f_{01} + f_{00})$
- **Asymmetric Binary:** One value (usually 1) is more important/rare.

- Example: Disease Test (Positive=1, Negative=0) — positive is the rare, important outcome.
- When measuring similarity, only 1-1 matches count; 0-0 matches are ignored.
- **Formula:** Jaccard Coefficient = $f_{11} / (f_{11} + f_{10} + f_{01})$

Discrete (Numeric) — Countable, whole numbers.

- Examples: Number of children (0, 1, 2, 3), Number of rooms (1, 2, 3).
- You CAN do math: Average number of children = 2.3.

Continuous (Numeric) — Can take any value in a range, including decimals.

- Examples: Height (5.7 ft), Weight (65.3 kg), Temperature (36.6°C).
- You CAN do all math: mean, std, variance, etc.

2.2 Basic Statistical Descriptions

Statistics help us understand and summarize data. There are two key groups:

A. Measures of Central Tendency (Where is the "center"?)

1. Mean (Average)

- Formula: $\mu = \Sigma x_i / N$
- Example: Data = {10, 20, 30, 40, 50} → Mean = 150/5 = **30**
- **Sensitive to outliers:** If data is {10, 20, 30, 40, 500} → Mean = 120 (pulled up by 500).

2. Median (Middle Value)

- Sort the data, pick the middle value.
- Odd count: Middle element directly. {10, 20, **30**, 40, 50} → Median = **30**
- Even count: Average of two middle elements. {10, 20, 30, 40} → Median = (20+30)/2 = **25**
- **Robust to outliers:** {10, 20, 30, 40, 500} → Median = **30** (unaffected!)

3. Mode (Most Frequent)

- The value that appears most often.
- {10, 20, 20, 30, 40} → Mode = **20** (appears twice).
- A dataset can have no mode (all unique), one mode (unimodal), or multiple modes (bimodal, multimodal).

When to use which?

Situation	Use
Normal distribution, no outliers	**Mean**
Skewed data or outliers present	**Median**
Categorical data	**Mode**

B. Measures of Dispersion (How "spread out" is the data?)

1. Range

- Simplest measure: Range = Max − Min
- {10, 20, 30, 40, 50} → Range = 50 − 10 = **40**
- Problem: Uses only 2 values, ignores everything in between.

2. Quartiles (Q1, Q2, Q3) Quartiles divide sorted data into 4 equal parts:

- **Q1 (25th percentile):** 25% of data is below this value.
- **Q2 (50th percentile):** Same as the Median.
- **Q3 (75th percentile):** 75% of data is below this value.

Example: Data (sorted) = {5, 10, 15, 20, 25, 30, 35}

- Q1 = Median of lower half {5, 10, 15} = **10**

- Q2 = Median of all = **20**
- Q3 = Median of upper half {25, 30, 35} = **30**

3. Interquartile Range (IQR)

- IQR = Q3 – Q1 = 30 – 10 = **20**
- Represents the middle 50% of data.
- **Key use: Outlier Detection!**
  - Lower fence = Q1 – 1.5 × IQR
  - Upper fence = Q3 + 1.5 × IQR
  - Any value outside these fences = **Outlier**

4. Variance (σ^2)

- Average of squared deviations from the mean.
- Formula: $\sigma^2 = \Sigma(x_i - \mu)^2 / N$
- Tells you how much data varies from the average.

5. Standard Deviation (σ)

- Square root of variance: $\sigma = \sqrt{(\sigma^2)}$
- In the **same units** as the data (unlike variance which is in squared units).
- Small σ = data points are close to the mean. Large σ = data is spread out.

Worked Example:

Data = {4, 8, 6, 5, 3}

Step 1: Mean = (4+8+6+5+3)/5 = 26/5 = **5.2**

Step 2: Deviations:

x_i	$x_i - \mu$	$(x_i - \mu)^2$
4	−1.2	1.44
8	2.8	7.84
6	0.8	0.64
5	−0.2	0.04
3	−2.2	4.84
Sum		**14.80**

Step 3: Variance = 14.80 / 5 = **2.96**

Step 4: Standard Deviation = √2.96 = **1.72**

2.3 Data Preprocessing

Raw data is ALMOST ALWAYS messy. Preprocessing transforms it into a usable format.

Why is Preprocessing Needed?

- Real data has missing values, duplicates, errors, noise.
- ML models can't work with NaN values.
- Features at different scales (age: 0-100, salary: 0-1000000) cause problems.
- Clean data → Better models → Better predictions.

The Four Steps:

Step 1: Data Cleaning

a) Handling Missing Values

Why data goes missing: Sensor malfunction, user skipped a field, data entry error.

Method	How it works	When to use	Code
Delete rows	Remove entire row with missing value	Missing % is very small (<5%)	`df.dropna()`
Delete columns	Remove entire column	Column has >50% missing	`df.drop(columns=['col'])`
Fill with Mean	Replace NaN with column average	Numeric, normally distributed	`df['col'].fillna(df['col'].mean())`
Fill with Median	Replace NaN with column median	Numeric, skewed data or outliers	`df['col'].fillna(df['col'].median())`
Fill with Mode	Replace NaN with most frequent value	Categorical data	`df['col'].fillna(df['col'].mode()[0])`
Forward Fill (ffill)	Use previous valid value	Time-series data	`df['col'].ffill()`
Backward Fill (bfill)	Use next valid value	Time-series data	`df['col'].bfill()`
Interpolation	Estimate based on surrounding values	Continuous, smooth data	`df['col'].interpolate()`

Types of Missingness (Advanced):

- **MCAR (Missing Completely At Random):** Missingness has no pattern — purely random. Safe to delete.
- **MAR (Missing At Random):** Missingness depends on another observed variable. E.g., Income is missing more for low-education people. Use conditional imputation.
- **MNAR (Missing Not At Random):** Missingness depends on the missing value itself. E.g., high-income people refuse to report income. Hardest to handle — need domain knowledge.

b) Handling Noisy Data

Noise = random errors in data (typos, sensor glitches).

Method	How it works
Binning	Sort data, divide into bins, smooth by bin mean/median/boundary
Regression	Fit data to a smooth function
Clustering	Group similar data, outliers don't fit any group
Manual inspection	Domain experts review flagged values

c) Removing Duplicates

```
df = df.drop_duplicates()                    # Remove exact duplicates
df = df.drop_duplicates(subset='Email')      # Remove duplicates based on email
```

d) Resolving Inconsistencies

- "Male", "M", "male", "MALE" → Standardize to "M"
- "NY", "New York", "new york" → Standardize to "New York"

```
df['Gender'] = df['Gender'].str.upper().str[0]  # All become M or F
df['City'] = df['City'].str.strip().str.title()  # "  new york " → "New York"
```

Step 2: Data Integration

Combining data from **multiple sources** into one unified dataset.

- Challenge: Different schemas, naming conventions, keys.
- Example: Merge CRM data (Customer_ID, Name, Email) with Sales data (Cust_ID, Product, Amount).
- Requires matching `Customer_ID` with `Cust_ID`.

```
merged = pd.merge(crm_df, sales_df, left_on='Customer_ID', right_on='Cust_ID', how='inner')
```

Step 3: Data Transformation

Converting data into a suitable format for analysis.

a) Normalization (Scaling)

Why needed: If Age is 0-100 and Salary is 0-1000000, salary will dominate in distance calculations. Normalization brings everything to the same scale.

Min-Max Normalization: Scales to [0, 1]

```
X_normalized = (X − X_min) / (X_max − X_min)
```

Example: Data={10, 20, 30, 40, 50}, for X=30: (30–10)/(50–10) = 20/40 = **0.5**

Z-Score Standardization: Centers around mean=0, std=1

```
Z = (X − μ) / σ
```

Example: μ=30, σ=14.14, for X=45: (45–30)/14.14 = **1.06**

Robust Scaling: Uses median and IQR (outlier-resistant)

```
X_robust = (X − Median) / IQR
```

Method	Best for	Outlier-resistant?
Min-Max	Bounded, clean data	× No
Z-Score	Normal distribution	× Moderate
Robust	Data with outliers	✅ Yes

b) Encoding Categorical Data

ML models need numbers, not text. We convert categories to numbers.

Label Encoding: Assign each category a number.

```
# Small→0, Medium→1, Large→2
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['Size_encoded'] = le.fit_transform(df['Size'])
```

⚠️ Problem: Model might think Large (2) > Small (0). Use only for ordinal data.

One-Hot Encoding: Create binary columns for each category.

```
df_encoded = pd.get_dummies(df['City'], dtype=int)
# Delhi→[1,0,0], Mumbai→[0,1,0], Pune→[0,0,1]
```

✅ No artificial ordering. Use for nominal data.

c) Aggregation: Summarize data at a higher level.

- Daily sales → Monthly totals.
- Individual student marks → Department averages.

d) Discretization (Binning): Convert continuous → categorical.

- Age: 0-18→Child, 19-60→Adult, 60+→Senior.
- **Equal-Width:** Each bin covers same range. Width = (max−min)/num_bins.
- **Equal-Frequency (Depth):** Each bin has same number of values.

Step 4: Data Reduction

Making the dataset smaller while preserving important information.

a) Dimensionality Reduction — Reduce the number of features.

- **PCA (Principal Component Analysis):** Mathematically creates new features (principal components) that capture maximum variance. 100 features → 10 components retaining 95% information.
- **Feature Selection:** Pick only the most relevant features using correlation analysis, chi-square test, or domain knowledge.

b) Numerosity Reduction — Reduce the number of data points.

- **Sampling:** Take a representative subset (random, stratified, systematic).
- **Parametric Models:** Store only model parameters instead of raw data.

c) Data Compression

- **Lossless:** No information lost (ZIP, gzip). Can perfectly reconstruct original.
- **Lossy:** Some information lost (JPEG, MP3). Approximate reconstruction.

2.4 Ethical Considerations in Data Science

1. Data Privacy

- People have the right to know what data is collected about them.
- **GDPR** (Europe): Requires explicit consent, right to be forgotten.
- **DPDP Act** (India, 2023): India's data protection law.
- Example: Fitness apps collecting location data without telling users.

2. Algorithmic Bias

- Models can discriminate based on race, gender, or socioeconomic status.
- **Example:** Amazon's AI hiring tool (2018) — biased against women because it was trained on 10 years of male-dominated resumes.
- Fix: Balanced training data, fairness audits, diverse teams.

3. Informed Consent

- Users must clearly understand and agree to data collection.
- **Example:** Cambridge Analytica (2018) — harvested 87M Facebook profiles without proper consent for political profiling.

4. Transparency

- AI decisions should be explainable, not "black boxes."
- Tools: LIME, SHAP for explaining model predictions.

5. Data Security

- Encrypt data, control access, regular security audits.
- **Example:** Equifax breach (2017) — 147 million people's personal data exposed.

2.5 Machine Learning for Data Science

What is Machine Learning?

ML is a subset of AI where computers **learn from data** without being explicitly programmed. Instead of writing rules, you give the machine data and it discovers the rules.

Types of Machine Learning:

1. Supervised Learning

- **Data:** Labelled (input + correct output provided).
- **Goal:** Learn the mapping from input → output so you can predict output for new inputs.
- **Types:**
  - **Classification** — Output is a category. (Spam/Not Spam, Cat/Dog)
  - **Regression** — Output is a number. (House price = ₹45L, Temperature = 32°C)
- **Algorithms:** k-NN, Decision Tree, Linear Regression, SVM, Naive Bayes.
- **Example:** Training a model on 10,000 emails labelled as spam/not spam → model predicts spam for new emails.

2. Unsupervised Learning

- **Data:** Unlabelled (only inputs, no correct answers).
- **Goal:** Discover hidden patterns, groupings, or structure in data.
- **Types:**
  - **Clustering** — Group similar items. (Customer segments)
  - **Association** — Find rules. ("Buy bread → buy butter")
  - **Dimensionality Reduction** — PCA
- **Algorithms:** K-Means, DBSCAN, Apriori, PCA.
- **Example:** Grouping customers into segments (budget, premium, luxury) without predefined labels.

3. Semi-Supervised Learning

- **Data:** Small amount of labelled + large amount of unlabelled.
- **Why needed:** Labelling is expensive (requires human experts, like doctors labelling X-rays).
- **How it works:** Learn from the few labelled examples, then use that knowledge to classify unlabelled data.
- **Example:** Google Photos — you label a few faces, and the system automatically identifies those people across thousands of photos.

4. Reinforcement Learning

- **Data:** No dataset! Agent interacts with an environment.
- **How it works:** Agent takes actions → receives rewards (good action) or penalties (bad action) → learns to maximize total reward.
- **Key Concepts:** Agent, State, Action, Reward, Policy.
- **Example:** AlphaGo (DeepMind) — learned to play Go by playing millions of games against itself.

Comparison Table:

Aspect	Supervised	Unsupervised	Semi-supervised	Reinforcement
Data	Labelled	Unlabelled	Few labelled + many unlabelled	No data (environment)
Feedback	Direct (correct answer)	None	Partial	Reward/penalty
Goal	Predict output	Find patterns	Predict with few labels	Maximize reward
Example	Spam detection	Customer segmentation	Medical imaging	Game AI

2.6 k-Nearest Neighbours (k-NN)

What is k-NN?

k-NN is a **supervised, lazy, instance-based** algorithm used for classification and regression.

- **Lazy** = It does NOT learn during training. It stores all data and computes at prediction time.
- **Instance-based** = It compares new data points to existing instances.

How it Works:

1. **Choose k** (number of neighbours to consider).
2. **Calculate distance** from the new point to ALL training points.
3. **Select the k nearest** points (with smallest distances).
4. **Classification:** Majority vote among k neighbours.

5. **Regression:** Average value of k neighbours.

Distance Metrics:

Euclidean Distance (most common):

$$d = \sqrt{[(x_1-y_1)^2 + (x_2-y_2)^2 + ... + (x_n-y_n)^2]}$$

Manhattan Distance (sum of absolute differences):

$$d = |x_1-y_1| + |x_2-y_2| + ... + |x_n-y_n|$$

Minkowski Distance (generalized):

$$d = (\Sigma|x_i-y_i|^p)^{(1/p)}$$

- p=1 → Manhattan, p=2 → Euclidean.

Effect of k:

- **k too small (e.g., k=1):** Overfitting. Sensitive to noise. One noisy point can change the prediction.
- **k too big (e.g., k=100):** Underfitting. Ignores local patterns. Everything gets classified as the majority class.
- **Best practice:** Use odd k (to avoid ties). Find optimal k using cross-validation.

Worked Example:

Training data:

Point	X	Y	Class
A	1	2	Cat
B	2	3	Cat
C	6	7	Dog
D	7	8	Dog
E	3	3	Cat

New point P = (4, 5), k = 3.

Distances:

- $d(P,A) = \sqrt{((4-1)^2+(5-2)^2)} = \sqrt{(9+9)} = \sqrt{18} =$ **4.24**
- $d(P,B) = \sqrt{((4-2)^2+(5-3)^2)} = \sqrt{(4+4)} = \sqrt{8} =$ **2.83**
- $d(P,C) = \sqrt{((4-6)^2+(5-7)^2)} = \sqrt{(4+4)} = \sqrt{8} =$ **2.83**
- $d(P,D) = \sqrt{((4-7)^2+(5-8)^2)} = \sqrt{(9+9)} = \sqrt{18} =$ **4.24**
- $d(P,E) = \sqrt{((4-3)^2+(5-3)^2)} = \sqrt{(1+4)} = \sqrt{5} =$ **2.24**

3 nearest: E (Cat, 2.24), B (Cat, 2.83), C (Dog, 2.83) Majority: 2 Cat, 1 Dog → **Predicted: Cat** ✅

2.7 K-Means Clustering

What is K-Means?

K-Means is an **unsupervised** algorithm that partitions data into **K groups (clusters)** where each point belongs to the cluster with the nearest centroid (center).

Algorithm Steps:

1. **Choose K** (number of clusters).
2. **Initialize** K centroids (randomly or given).
3. **Assign** each data point to the nearest centroid → forms K clusters.
4. **Update** centroids = mean of all points in that cluster.

5. **Repeat** steps 3-4 until centroids stop changing (**convergence**).

Worked Example:

Data: {2, 3, 8, 9, 10}, K=2

Initial centroids: C1=2, C2=10

Iteration 1 — Assign:

| Point | |d to C1=2| | |d to C2=10| | Cluster | |:---:|:---:|:---:|:---:| | 2 | 0 | 8 | C1 | | 3 | 1 | 7 | C1 | | 8 | 6 | 2 | C2 | | 9 | 7 | 1 | C2 | | 10 | 8 | 0 | C2 |

Update centroids:

- C1 = (2+3)/2 = **2.5**
- C2 = (8+9+10)/3 = **9.0**

Iteration 2 — Assign (C1=2.5, C2=9.0):

| Point | |d to C1=2.5| | |d to C2=9.0| | Cluster | |:---:|:---:|:---:|:---:| | 2 | 0.5 | 7.0 | C1 | | 3 | 0.5 | 6.0 | C1 | | 8 | 5.5 | 1.0 | C2 | | 9 | 6.5 | 0.0 | C2 | | 10 | 7.5 | 1.0 | C2 |

Same clusters as before → **Converged!** ✅ Final: Cluster 1 = {2, 3}, Cluster 2 = {8, 9, 10}

2.8 Outlier Detection

What are Outliers?

Data points that are significantly different from the rest. Example: In salaries {30K, 35K, 40K, 45K, 500K}, the 500K is an outlier.

Method 1: IQR Method

- Q1 = 25th percentile, Q3 = 75th percentile
- IQR = Q3 – Q1
- **Lower fence** = Q1 – 1.5 × IQR
- **Upper fence** = Q3 + 1.5 × IQR
- Anything outside fences = Outlier

Robust — Q1 and Q3 are not affected by extreme values.

Method 2: Z-Score Method

- $Z = (X - \mu) / \sigma$
- If $|Z| > 3$ → Outlier (the point is 3+ standard deviations from mean)

Not robust — Mean and σ are themselves distorted by outliers.

Comparison:

	IQR	Z-Score
Robust?	✅ Yes	× No
Best for	Skewed data	Normal distributions
Threshold	1.5 × IQR	Z > 3

2.9 Correlation

Correlation measures the strength and direction of the relationship between two variables.

Pearson Correlation Coefficient (r):

$$r = \Sigma(x_i - \bar{x})(y_i - \bar{y}) \;/\; \sqrt{[\Sigma(x_i - \bar{x})^2 \times \Sigma(y_i - \bar{y})^2]}$$

Value of r	Meaning
+1	Perfect positive correlation (as X↑, Y↑)
+0.7 to +1	Strong positive
+0.3 to +0.7	Moderate positive
0	No correlation
–0.3 to –0.7	Moderate negative
–0.7 to –1	Strong negative
–1	Perfect negative correlation (as X↑, Y↓)

2.10 Model Evaluation Metrics

When you build a model, how do you know if it's good?

Confusion Matrix:

	Predicted Positive	Predicted Negative
Actual Positive	TP (True Positive)	FN (False Negative)
Actual Negative	FP (False Positive)	TN (True Negative)

Metrics:

Metric	Formula	What it measures	When to prioritize
Accuracy	(TP+TN) / Total	Overall correctness	Balanced datasets
Precision	TP / (TP+FP)	Of predicted positives, how many are correct?	When false alarms are costly (spam detection)
Recall	TP / (TP+FN)	Of actual positives, how many were caught?	When missing positives is dangerous (disease detection)
F1-Score	2×(P×R)/(P+R)	Harmonic mean of precision & recall	Imbalanced datasets
Specificity	TN / (TN+FP)	Of actual negatives, how many were correctly identified?	Important in medical screening

Class Imbalance Problem:

If 95% of data is class A and 5% is class B, a model that always predicts A gets 95% accuracy — but it's useless for detecting B!

Solutions:

- **SMOTE** — Create synthetic minority samples.
- **Class Weights** — Penalize misclassification of minority class more.
- **Undersampling** — Reduce majority class.
- **Use F1-Score** instead of accuracy.

2.11 Sampling Techniques

Technique	How it works	Example	Pros	Cons
Simple Random	Every item has equal chance	Draw 100 names from a hat	Unbiased	May miss subgroups

Systematic	Select every k-th item	Every 10th product on assembly line	Easy	Bias if data has cycles
Stratified	Divide into groups (strata), sample proportionally	Sample students from each department	All groups represented	Need to know strata
Cluster	Divide into clusters, randomly pick entire clusters	Survey all households in 5 random villages	Cost-effective	High variability
Convenience	Use whatever is easily available	Survey friends on campus	Quick	Highly biased

---

🔷 UNIT 3: PYTHON FOR DATA ANALYSIS & VISUALIZATION

3.1 Introduction to Python (for Data Science)

Python is the most popular language for data science because:

- Simple, readable syntax.
- Massive ecosystem of libraries (NumPy, Pandas, Matplotlib, Scikit-learn).
- Large community and free resources.

Key Python Concepts for Data Science:

Variables and Types:

```
x = 10          # int
y = 3.14        # float
name = "Rajat"  # string
is_student = True  # boolean
```

Lists (mutable, ordered):

```
fruits = ["apple", "banana", "cherry"]
fruits.append("mango")       # Add item
fruits[0]                    # Access: "apple"
len(fruits)                  # Length: 4
```

Tuples (immutable, ordered):

```
point = (10, 20)
# point[0] = 30  ← ERROR! Can't modify tuples
```

Dictionaries (key-value pairs):

```
student = {"name": "Rajat", "age": 21, "cgpa": 8.5}
student["name"]              # "Rajat"
student["city"] = "Delhi"    # Add new key
```

Functions:

```
def sum_list(numbers):
    return sum(numbers)

result = sum_list([1, 2, 3, 4, 5])  # 15
```

Operator Precedence (highest to lowest):

1. `**` (exponentiation)
2. `*`, `/`, `//`, `%` (left to right)
3. `+`, `-` (left to right)

Example: `x=7, y=3, y = x//y = 2`, then `z = 7**2 // 2 + 7%2 * 3 = 49//2 + 1*3 = 24+3 = 27`

3.2 NumPy — Numerical Python

NumPy provides fast, efficient arrays for numerical computations.

Why NumPy over Python lists?

- **Speed:** NumPy arrays are stored in contiguous memory → 50-100x faster.
- **Functionality:** Built-in math functions (mean, std, dot product, matrix operations).
- **Broadcasting:** Operations on arrays of different shapes.

```
import numpy as np

# Create arrays
arr = np.array([10, 20, 30, 40, 50])
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# Basic operations
print(np.mean(arr))       # 30.0
print(np.median(arr))     # 30.0
print(np.std(arr))        # 14.14
print(np.var(arr))        # 200.0
print(np.min(arr))        # 10
print(np.max(arr))        # 50
print(np.sum(arr))        # 150
print(np.argmax(arr))     # 4 (index of max value)

# Reshape
arr2 = np.arange(1, 10)          # [1, 2, 3, ..., 9]
matrix = arr2.reshape(3, 3)     # 3x3 matrix

# Element-wise operations
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(a + b)    # [5, 7, 9]
print(a * b)    # [4, 10, 18]
print(a ** 2)   # [1, 4, 9]

# Column/Row sums
print(np.sum(matrix, axis=0))   # Column-wise sum
print(np.sum(matrix, axis=1))   # Row-wise sum

# Random numbers
random_arr = np.random.normal(50, 10, 100)  # 100 values, mean=50, std=10
```

3.3 Pandas — Data Analysis

Pandas is the most important library for data science. It provides two key data structures:

Series (1D labelled array):

```
import pandas as pd
```

```
s = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
print(s['b'])  # 20
```

DataFrame (2D labelled table):

```
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [22, 25, 23],
    'Marks': [85, 90, 78]
}
df = pd.DataFrame(data)
```

Essential Pandas Operations:

Reading Data:

```
df = pd.read_csv('data.csv')          # Read CSV
df = pd.read_excel('data.xlsx')       # Read Excel
df = pd.read_json('data.json')        # Read JSON
```

Exploring Data:

```
df.head()           # First 5 rows
df.tail()           # Last 5 rows
df.shape            # (rows, columns)
df.dtypes           # Data types
df.describe()       # Statistical summary
df.info()           # Compact info
df.columns.tolist() # List of column names
df.nunique()        # Unique values per column
```

Handling Missing Values:

```
df.isnull().sum()                        # Count NaN per column
df['col'].fillna(df['col'].mean())       # Fill with mean
df['col'].fillna(df['col'].median())     # Fill with median
df['col'].fillna(df['col'].mode()[0])    # Fill with mode
df.dropna()                               # Drop rows with any NaN
df.dropna(subset=['col1', 'col2'])       # Drop if specific cols are NaN
df.ffill()                                # Forward fill
df.bfill()                                # Backward fill
df['col'].interpolate()                   # Linear interpolation
```

Selecting & Filtering:

```
df['Name']                                # Select one column
df[['Name', 'Age']]                       # Select multiple columns
df[df['Age'] > 25]                        # Filter rows
df[(df['Age'] > 20) & (df['Marks'] > 80)] # Multiple conditions
df.loc['a', 'Name']                       # Label-based
df.iloc[0, 0]                             # Position-based
```

`loc` vs `iloc`:

	`loc`	`iloc`
Type	Label-based	Integer position-based

End	Inclusive	Exclusive
Example	`df.loc['a':'c']` → includes c	`df.iloc[0:3]` → excludes index 3

Adding/Modifying Columns:

```
df['City'] = ['Delhi', 'Mumbai', 'Pune']    # Add new column
df['Pass'] = df['Marks'] > 80              # Computed column
df['Grade'] = df['Marks'].apply(lambda x: 'A' if x > 85 else 'B')
```

Sorting:

```
df.sort_values('Marks', ascending=False)    # Sort by marks descending
df.sort_values(['Dept', 'Marks'])          # Multi-column sort
```

Grouping & Aggregation:

```
df.groupby('Department')['Salary'].mean()              # Average salary per dept
df.groupby('Department').agg({'Salary': ['mean', 'max', 'min']})  # Multiple aggs
df.groupby(['Department', 'Gender'])['Salary'].mean()  # Multi-level grouping
```

Removing Duplicates:

```
df = df.drop_duplicates()
df = df.drop_duplicates(subset='Email', keep='first')
```

Saving Data:

```
df.to_csv('output.csv', index=False)
df.to_excel('output.xlsx', index=False)
```

Pivot Tables:

```
pivot = df.pivot_table(values='Salary', index='Department', columns='Gender', aggfunc='mean')
```

Merging DataFrames:

```
merged = pd.merge(df1, df2, on='ID', how='inner')      # Inner join
merged = pd.merge(df1, df2, on='ID', how='left')       # Left join
```

3.4 Matplotlib — Basic Plotting

Matplotlib is the foundation of data visualization in Python.

Line Chart:

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [10, 20, 25, 30, 40]

plt.figure(figsize=(8, 5))
plt.plot(x, y, marker='o', color='blue', linestyle='-')
plt.title('Line Chart')
plt.xlabel('X-Axis')
```

```
plt.ylabel('Y-Axis')
plt.grid(True)
plt.show()
```

Bar Chart:

```
categories = ['CSE', 'ECE', 'ME']
values = [85, 78, 72]

plt.bar(categories, values, color=['green', 'blue', 'red'])
plt.title('Average Marks by Department')
plt.show()
```

Histogram (Distribution):

```
data = np.random.normal(50, 10, 500)

plt.hist(data, bins=20, color='steelblue', edgecolor='black', alpha=0.7)
plt.title('Score Distribution')
plt.xlabel('Score')
plt.ylabel('Frequency')
plt.show()
```

Scatter Plot (Correlation):

```
plt.scatter(df['Hours'], df['Marks'], color='coral', alpha=0.6)
plt.title('Study Hours vs Marks')
plt.xlabel('Study Hours')
plt.ylabel('Marks')
plt.show()
```

Pie Chart:

```
labels = ['CSE', 'ECE', 'ME', 'CE']
sizes = [40, 25, 20, 15]
plt.pie(sizes, labels=labels, autopct='%1.1f%%')
plt.title('Department Distribution')
plt.show()
```

Subplots (Multiple Charts):

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].bar(categories, values)
axes[0].set_title('Bar Chart')
axes[1].hist(data, bins=15)
axes[1].set_title('Histogram')
plt.tight_layout()
plt.show()
```

When to Use Each Chart:

Chart	Best for	Example
Line	Trends over time	Stock price over months

Bar	Comparing categories	Sales by region
Histogram	Distribution of one variable	Age distribution
Scatter	Relationship between 2 variables	Height vs Weight
Pie	Proportions of a whole	Market share
Boxplot	Spread, median, outliers	Salary distribution by dept

3.5 Seaborn — Statistical Visualization

Seaborn is built on Matplotlib and provides **high-level, beautiful, statistical plots** with less code.

Key Difference from Matplotlib:

- Seaborn works directly with Pandas DataFrames.
- Has built-in themes and color palettes.
- Statistical plots (boxplot, violin, heatmap) are much easier.

Boxplot:

```
import seaborn as sns

sns.boxplot(x='Department', y='Salary', data=df, palette='Set2')
plt.title('Salary by Department')
plt.show()
```

Reading a Boxplot:

- **Box:** Q1 to Q3 (middle 50%).
- **Line inside box:** Median (Q2).
- **Whiskers:** Extend to 1.5×IQR from box.
- **Dots beyond whiskers:** Outliers.

Heatmap (Correlation Matrix):

```
correlation = df.corr()
sns.heatmap(correlation, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap')
plt.show()
```

Histogram with KDE:

```
sns.histplot(df['Marks'], kde=True, color='teal')
plt.title('Marks Distribution')
plt.show()
```

Violin Plot:

```
sns.violinplot(x='Department', y='Salary', data=df, palette='muted')
plt.title('Salary Violin Plot')
plt.show()
```

- Combines boxplot + KDE (density estimation).
- Shows the full distribution shape, not just summary statistics.

Pair Plot:

```
sns.pairplot(df[['Age', 'Salary', 'Rating']], diag_kind='kde')
plt.show()
```

- Shows scatter plots for every pair of variables.
- Diagonal shows distribution of each variable.
- Great for spotting correlations quickly.

Count Plot:

```
sns.countplot(x='Department', hue='Gender', data=df)
plt.title('Gender Distribution per Department')
plt.show()
```

3.6 Folium — Spatial Visualization

Folium creates **interactive maps** in Python using Leaflet.js.

Basic Map:

```
import folium

m = folium.Map(location=[28.6139, 77.2090], zoom_start=12)
m.save('map.html')
```

Add Markers:

```
folium.Marker(
    location=[28.6139, 77.2090],
    popup='New Delhi',
    tooltip='Click for details',
    icon=folium.Icon(color='red', icon='info-sign')
).add_to(m)
```

Circle Markers (proportional to data):

```
folium.CircleMarker(
    location=[28.6139, 77.2090],
    radius=30,  # proportional to population
    color='blue',
    fill=True,
    fill_opacity=0.5
).add_to(m)
```

Marker Clustering (for many points):

```
from folium.plugins import MarkerCluster
mc = MarkerCluster().add_to(m)
folium.Marker([lat, lon]).add_to(mc)
```

🔷 BONUS: ADVANCED CONCEPTS (From Question Papers)

B.1 Association Rule Mining

Used to find relationships like "customers who buy X also buy Y."

Key Metrics:

Metric	Formula	Meaning
Support	freq(A∩B) / N	How often A and B appear together
Confidence	freq(A∩B) / freq(A)	Probability of B given A was bought
Lift	Confidence / Support(B)	Improvement over random chance

- **Lift > 1:** A and B are positively associated.
- **Lift = 1:** Independent.
- **Lift < 1:** Negatively associated.

B.2 Data Governance Framework

A framework for managing data across an organization:

Layer	What it covers
Policy	Data classification, retention, access control, privacy policies
Process	Data cataloging, quality monitoring, change management, auditing
Technology	Encryption, data lakes, monitoring tools, version control
People	Data stewards, training, ethics boards
Compliance	GDPR, DPDP Act, RBI guidelines, SEBI regulations

B.3 Data Drift and Concept Drift

Data Drift: Input data distribution changes, but the relationship between features and target stays the same.

- Example: A model trained on pre-COVID spending patterns sees very different spending after COVID.

Concept Drift: The actual relationship between features and target changes.

- Example: What customers consider "fast delivery" changes from 3 days (2019) to same-day (2025).

Both cause model performance to degrade over time → need monitoring and retraining.

B.4 The Curse of Dimensionality

As the number of features increases:

- Data becomes increasingly **sparse** (points are far apart in high dimensions).
- **All distances converge** — nearest and farthest points become equidistant.
- Models like k-NN and K-Means (which rely on distance) break down.
- **Solution:** Reduce dimensions using PCA or feature selection.

B.5 Interpretability vs Accuracy Trade-off

Simple models (Linear Regression) = Easy to understand but less accurate. Complex models (Deep Learning) = Highly accurate but "black box."

When to choose simple: Regulated industries (banking, healthcare), need to explain decisions. **When complex is OK:** Image recognition, NLP, when accuracy gap is large. **Tools for explaining complex models:** LIME, SHAP.

📝 Quick Revision — Key Formulas

Concept	Formula
Mean	$\mu = \Sigma x_i / N$
Variance	$\sigma^2 = \Sigma(x_i-\mu)^2 / N$
Std Deviation	$\sigma = \sqrt{(\text{Variance})}$
IQR	Q3 – Q1
Outlier (IQR)	$x < Q1-1.5\times IQR$ or $x > Q3+1.5\times IQR$
Z-Score	$Z = (X - \mu) / \sigma$
Min-Max	(X – Xmin) / (Xmax – Xmin)
Pearson r	$\Sigma(x-\bar{x})(y-\bar{y}) / \sqrt{[\Sigma(x-\bar{x})^2 \cdot \Sigma(y-\bar{y})^2]}$
Euclidean dist	$\sqrt{[\Sigma(x_i-y_i)^2]}$
Manhattan dist	Σ
SMC	$(f_{11}+f_{00}) / (f_{11}+f_{10}+f_{01}+f_{00})$
Jaccard	$f_{11} / (f_{11}+f_{10}+f_{01})$
Accuracy	(TP+TN) / Total
Precision	TP / (TP+FP)
Recall	TP / (TP+FN)
F1-Score	2×P×R / (P+R)
Support	freq(A∩B) / N
Confidence	freq(A∩B) / freq(A)
Lift	Confidence / Support(B)

📝 Quick Revision — Python Commands

Task	Code
Read CSV	`pd.read_csv('file.csv')`
Shape	`df.shape`
First 5 rows	`df.head()`
Data types	`df.dtypes`
Missing values	`df.isnull().sum()`
Fill NaN mean	`df['col'].fillna(df['col'].mean())`
Drop NaN	`df.dropna()`
Drop duplicates	`df.drop_duplicates()`
Filter	`df[df['col'] > value]`
Group by	`df.groupby('col')['target'].mean()`
Sort	`df.sort_values('col', ascending=False)`

New column	`df['new'] = df['a'] + df['b']`
Apply function	`df['col'].apply(func)`
Save CSV	`df.to_csv('out.csv', index=False)`
Histogram	`plt.hist(data, bins=15)`
Boxplot	`sns.boxplot(x='cat', y='num', data=df)`
Scatter	`plt.scatter(x, y)`
Heatmap	`sns.heatmap(df.corr(), annot=True)`
NumPy mean	`np.mean(arr)`
NumPy std	`np.std(arr)`

— End of Study Material —

> 💡 ***Tip:*** *Read this material once fully, then solve the question papers. Revisit specific sections when you get stuck. Good luck with your exam!* 🎯